



Hanzo Edge: On-Device AI Inference for Mobile and IoT Deployments

Zach Kelling

Hanzo AI

July 2024

Abstract

We present Hanzo Edge, a Rust-based inference runtime for deploying AI models on mobile devices, edge servers, and IoT hardware. Edge supports ONNX model loading with automatic optimization for target hardware, including Apple Neural Engine (ANE) on iOS/macOS, GPU compute via Metal and Vulkan, and SIMD-accelerated CPU inference. The runtime implements dynamic quantization (INT8/INT4), operator fusion, and memory-mapped model loading to minimize memory footprint and startup latency. On an iPhone 15 Pro, Edge achieves 42 tokens per second for a 3B parameter language model and 8ms latency for a MobileNet-V3 image classifier. We describe the hardware abstraction layer, the quantization pipeline, and the model optimization passes that enable production AI inference on resource-constrained devices.

1 Introduction

On-device AI inference eliminates network latency, preserves user privacy, and enables offline operation. However, deploying models on edge devices requires navigating a fragmented hardware landscape: Apple Neural Engine, Qualcomm Hexagon DSP, ARM NEON, x86 AVX, and various GPU compute APIs. Existing runtimes (ONNX Runtime, TensorFlow Lite, Core ML) each target a subset of this landscape with varying levels of optimization.

Hanzo Edge provides a unified Rust runtime that accepts ONNX models and automatically selects the optimal execution backend for the target hardware. The Rust implementation ensures predictable performance without garbage collection pauses, while the C FFI enables integration with Swift (iOS), Kotlin (Android), and C++ (IoT) applications.

Contributions.

- A hardware abstraction layer supporting ANE, Metal, Vulkan, and SIMD CPU backends with automatic backend selection.

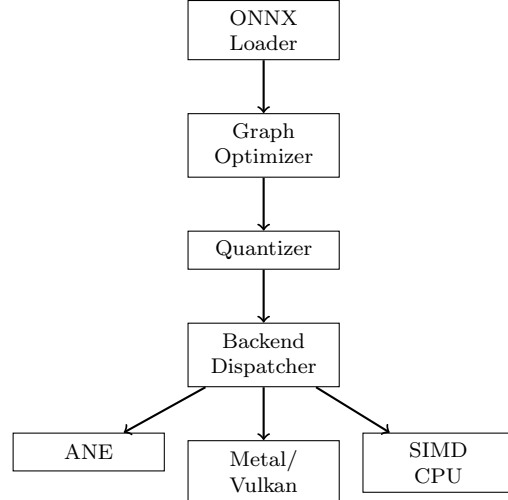


Figure 1: Edge inference pipeline from ONNX to hardware execution.

- Dynamic INT8 and INT4 quantization with calibration-free activation range estimation.
- Operator fusion passes that combine convolution-batchnorm-relu and attention patterns into single fused kernels.
- Memory-mapped model loading with lazy initialization, reducing cold-start time to under 50ms for models up to 3B parameters.

2 Architecture

2.1 Runtime Design

The Edge runtime processes models through a four-stage pipeline (Figure 1): ONNX graph loading, optimization passes, optional quantization, and hardware dispatch. Each stage produces an intermediate representation that the next stage consumes.

2.2 Hardware Abstraction Layer

The backend dispatcher selects the execution backend based on hardware capabilities and model requirements:

```
1 pub trait Backend: Send + Sync {  
2     fn name(&self) -> &str;
```

```

3     fn supports(&self, op: &OpType) ->
      bool;
4     fn execute(
5         &self,
6         graph: &OptimizedGraph,
7         inputs: &[Tensor],
8     ) -> Result<Vec<Tensor>>;
9     fn max_memory(&self) -> usize;
10 }

```

For heterogeneous execution, the dispatcher can split a graph across backends: attention layers on ANE, embedding lookups on CPU, and output projection on GPU. Graph partitioning uses a cost model calibrated per device family.

2.3 Model Format

Edge uses a custom binary format (.hze) that memory-maps weight tensors directly into the inference address space:

```

1 pub struct EdgeModel {
2     header: ModelHeader,    // 64 bytes
3     graph: ComputeGraph,    // operator
      DAG
4     weights: MmapSlice,     // mmap'd
      tensors
5     quantization: QuantInfo, // scale/
      zero-point
6 }

```

Memory mapping avoids copying weights into heap memory, reducing peak memory usage and enabling instant model loading.

3 Key Features

3.1 Dynamic Quantization

Edge supports post-training quantization without a calibration dataset:

INT8. Per-channel symmetric quantization for weights, per-tensor asymmetric for activations:

$$x_q = \text{clamp} \left(\text{round} \left(\frac{x}{s} \right) + z, 0, 255 \right) \quad (1)$$

where s is the scale factor and z is the zero point, computed from running min/max statistics.

INT4. Group-wise quantization with groups of 32 elements, achieving 8x compression with acceptable quality loss for language models.

3.2 Operator Fusion

The optimizer implements 12 fusion patterns:

- Conv + BatchNorm + ReLU → FusedConvBNReLU
- MatMul + Add + GELU → FusedLinearGELU
- Multi-Head Attention (Q/K/V projections + softmax + output) → FusedAttention
- LayerNorm + Linear → FusedNormLinear

Fused operators reduce memory bandwidth requirements by avoiding materialization of intermediate tensors.

3.3 Apple Neural Engine Support

On Apple Silicon, Edge compiles eligible sub-graphs to ANE via the Core ML backend. ANE is preferred for:

- Convolutions (2x–5x faster than GPU for common sizes)
- Matrix multiplications with batch size 1 (inference)
- Transformer attention blocks

Operations unsupported by ANE (custom activations, dynamic shapes) fall back to Metal GPU or NEON CPU.

4 Implementation

Edge is implemented in 22,000 lines of Rust with platform-specific backends: 3,000 lines of Metal shader code (Apple), 2,500 lines of Vulkan compute shaders (Android/Linux), and 1,500 lines of NEON/AVX intrinsics (CPU).

References

Table 1: Inference latency on iPhone 15 Pro.

Model	Params	Latency	Memory
MobileNet-V3	5.4M	8ms	22MB
Whisper-small	244M	1.2s/30s	480MB
Qwen3-3B (INT4)	3B	24ms/tok	1.8GB
CLIP-ViT-B/32	150M	18ms	310MB

- [1] ONNX. Open Neural Network Exchange. 2017.
- [2] B. Jacob et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR, 2018.
- [3] Apple. Core ML: Integrate machine learning models into your app. 2017.
- [4] A. Gholami et al. A Survey of Quantization Methods for Efficient Neural Network Inference. arXiv:2103.13630, 2022.

5 Security

Model Protection. Model weights can be encrypted with a device-specific key derived from the Secure Enclave (iOS) or Keystore (Android). The key never leaves the secure hardware.

Input Privacy. All inference runs on-device. No input data, intermediate activations, or results are transmitted to external servers.

Memory Safety. Rust’s ownership model prevents buffer overflows, use-after-free, and data races. The `unsafe` blocks are limited to hardware-specific FFI calls and SIMD intrinsics, totaling less than 5% of the codebase.

Sandboxed Execution. On iOS, Edge runs within the app sandbox. On Android, it uses SELinux confinement. On Linux/IoT, seccomp filters restrict system calls to file I/O and memory allocation.

6 Conclusion

Hanzo Edge enables production AI inference on resource-constrained devices by combining a Rust runtime with automatic hardware dispatch, dynamic quantization, and operator fusion. The memory-mapped model format and lazy initialization achieve sub-50ms cold start times, while INT4 quantization fits 3B parameter models within mobile memory constraints. By running inference entirely on-device, Edge preserves user privacy and eliminates network latency.